

PASUL 2 DIN 5

Cum funcționează loginul

Ghid pentru codul real al backendului Netex. Urmărim o cerere de autentificare de la email și parolă până la răspunsul HTTP.

Ideea principală: noi configurăm loginul și îi oferim lui Spring Security un mod de a citi utilizatorul din PostgreSQL. Spring primește cererea, compară parola cu hashul salvat și decide rezultatul.

După acest pas vei putea explica

- De ce nu există LoginController.java, deși backendul poate face login.
- Ce citește UserRepository din SQL și ce reprezintă AppUser.
- De ce AccountPrincipal implementează UserDetails.
- Cum se verifică o parolă fără a o salva sau a o compara în clar.
- Ce înseamnă răspunsurile 204, 401 și 403.

Stadiul proiectului: backendul de login este implementat și testat. Ruta React /login arată încă o pagină de pregătire; formularul va fi implementat în 5A.2.

Ordinea de citire

SecurityConfig.java → UserRepository.java → AppUser.java → AccountPrincipal.java →
SessionController.java → AuthApiTest.java

1. Unde este loginul?

În backend, loginul este configurat în SecurityConfig.java. Filtrul inclus în Spring Security preia cererea; de aceea folderul login poate fi gol și nu există LoginController.java.

SecurityConfig.java - fragment exact

```
.formLogin(login -> login
    .loginPage("/login")
    .loginProcessingUrl("/api/auth/login")
    .usernameParameter("email")
    .successHandler((request, response, authentication) -> response.setStatus(204))
    .failureHandler((request, response, error) -> response.sendError(401))
    .permitAll())
```

Linie cu linie

- formLogin activează mecanismul care primește câmpuri de formular.
- loginPage("/login") indică adresa paginii. Nu creează formularul React.
- loginProcessingUrl("/api/auth/login") spune la ce adresă POST verifică Spring datele.
- usernameParameter("email") spune că identicatorul vine din câmpul email.
- successHandler trimite 204, adică succes fără JSON în răspuns.
- failureHandler trimite 401 când datele de login sunt greșite.

Reține: /login este pagina. POST /api/auth/login este cererea procesată de backend. Nu sunt același lucru.

Cererea de login trimite câmpurile email și password în format application/x-www-form-urlencoded. Formularul React care va face această cerere nu este încă implementat.

2. Cum găsim utilizatorul?

Spring citește emailul din cerere și apelează UserDetailsService. Noi i-am dat această implementare în același SecurityConfig.java:

SecurityConfig.java - codul exact

```
@Bean
UserDetailsService userDetailsService(UserRepository users) {
    return email -> users.findByEmail(EmailAddress.normalize(email))
        .map(AccountPrincipal::from)
        .orElseThrow(() -> new UsernameNotFoundException("Unknown account"));
}
```

- @Bean face obiectul disponibil pentru Spring Security.
- email -> ... este funcția apelată pentru emailul din cerere.
- normalize taie spațiile și transformă literele în minuscule.
- findByEmail caută rândul din SQL. Încă nu verifică parola.
- map(AccountPrincipal::from) schimbă AppUser într-un obiect înțeles de Spring.
- orElseThrow înseamnă că emailul nu a fost găsit.

EmailAddress.java - codul exact

```
public static String normalize(String email) {
    return email.trim().toLowerCase(Locale.ROOT);
}
```

Interogarea din UserRepository.java

Textul SQL exact din UserRepository.java

```
SELECT id, email, password_hash, role FROM users WHERE lower(email) = lower(?)
```

Semnul ? primește emailul ca parametru prin JdbcTemplate. Rezultatul este transformat într-un AppUser. Optional<AppUser> arată că un cont poate exista sau poate lipsi.

3. Ce este AccountPrincipal?

UserRepository creează un AppUser cu id, email, passwordHash și role. Spring Security folosește interfața UserDetails, așa că transformăm acel AppUser în AccountPrincipal.

AppUser.java - codul exact

```
public record AppUser(long id, String email, String passwordHash, String role) {  
}
```

AccountPrincipal.java - începutul recordului

```
public record AccountPrincipal(long id, String email, String passwordHash, String role) implements UserDetails {  
    public static AccountPrincipal from(AppUser user) {  
        return new AccountPrincipal(user.id(), user.email(), user.passwordHash(), user.role());  
    }  
}
```

AccountPrincipal.java - metodele folosite la login

```
@Override  
public String getPassword() {  
    return passwordHash;  
}
```

```
@Override  
public String getUsername() {  
    return email;  
}
```

getUsername() oferă emailul. getPassword() oferă hashul citit din SQL, nu parola în clar. În același record, getAuthorities() transformă rolul USER sau ADMIN într-o autoritate Spring; vom studia permisiunile în pasul 4.

SecurityConfig.java - encoderul configurat

```
@Bean  
PasswordEncoder passwordEncoder() {  
    return new BCryptPasswordEncoder();  
}
```

Cine compară parola? Spring Security folosește PasswordEncoder ca să verifice parola introdusă față de hashul BCrypt. Codul nostru nu inversează hashul și nu compară parole în clar.

4. Fluxul complet, in ordinea executiei

1. Browserul trimite POST /api/auth/login cu email si password.
2. Filtrul de login din Spring Security preia cererea.
3. UserDetailsService normalizeaza emailul si cheama UserRepository.findByEmail.
4. PostgreSQL intoarce id, email, password_hash si role; codul face AppUser.
5. AppUser devine AccountPrincipal; Spring verifica parola cu BCryptPasswordEncoder.
6. La succes, Spring autentifica utilizatorul si raspunde 204. La date gresite, raspunde 401.

Signup nu inseamna login: POST /api/auth/signup creeaza contul in users, dar nu autentifica automat utilizatorul.

Cum aflu cine este autentificat?

SessionController.java - codul actual

```
@GetMapping("/me")
public UserResponse me(@AuthenticationPrincipal AccountPrincipal principal) {
    return UserResponse.from(principal);
}
```

GET /api/auth/me intoarce id, email si role dupa un login reusit. Fara autentificare raspunde 401. POST /api/auth/login raspunde 204 fara corp JSON.

Ce pastreaza autentificarea?

La login, Spring creeaza sau actualizeaza sesiunea HTTP. Browserul pastreaza cookie-ul de sesiune si il trimite automat la cererile urmatoare. Backendul gaseste sesiunea si stie cine este logat. In varianta de baza nu se cere token CSRF.

5. Cum verificam si ce exersezi

AuthApiTest.java include testul loginPersistsASessionAndLogoutInvalidatesIt. Citeste fragmentele in ordinea de mai jos.

AuthApiTest.java - pregatirea si verificarea initiala

```
jdbc.update("INSERT INTO users (email, password_hash) VALUES (?, ?)",
    email, passwordEncoder.encode("strong-password"));

mockMvc.perform(get("/api/auth/me")).andExpect(status().isUnauthorized());
```

Testul creeaza un rand cu hash BCrypt. Existenta contului nu inseamna ca persoana este deja logata: /me raspunde 401.

AuthApiTest.java - parola gresita

```
.param("email", email).param("password", "wrong-password"))
    .andExpect(status().isUnauthorized());
```

AuthApiTest.java - fragmente din loginul reusit

```
.param("email", email.toUpperCase())
.param("password", "strong-password"))
.andExpect(status().isNoContent())
```

Parola gresita produce 401, iar cea corecta produce 204. Emailul cu majuscule functioneaza datorita normalizarii si cautarii fara diferenta intre litere mari si mici.

Exercitiu de 15 minute

- In IntelliJ gaseste formLogin, UserDetailsService, findByEmail, getUsername si getPassword.
- In DBever observe users.email, users.password_hash si users.role, daca exista deja un cont.
- Explica singur: cine citeste contul, cine verifica parola si ce cod HTTP arata rezultatul?

Verificare rapida: De ce nu exista LoginController? Ce contine getPassword()? De ce /me raspunde 401 imediat dupa signup? Ce rol are cookie-ul de sesiune?

Urmatorul pas: sesiunea si cookie-ul. Protectia CSRF poate fi studiata dupa versiunea functionala a site-ului.